Smart Contract Security Audit Report

Gearbox Securitize Audit

1. Contents

2. General Information

This report contains information about the results of the security audit of the Gearbox (hereafter referred to as "Customer") smart contracts, conducted by Decurity in the period from 2026-04-27 to 2026-05-01.

2.1. Introduction

Tasks solved during the work are:

- Review the protocol design and the usage of 3rd party dependencies,
- Audit the contracts implementation,
- Develop the recommendations and suggestions to improve the security of the contracts.

2.2. Scope of Work

The audit scope included the contracts in the following repositories: https://github.com/Gearbox-protocol/integrations-v3/, https://github.com/Gearbox-protocol/periphery-v3/.

Initial review was done for the commits integrations-v3/commit/78d626 and periphery-v3/commits/403602 and the re-testing was done for the commits integrations-v3/commit/3be06c and periphery-v3/commit/c93857.

The following contracts have been tested:

- integrations-v3/contracts/adapters/securitize/SecuritizeOnRampAdapter.sol
- integrations-v3/contracts/adapters/securitize/SecuritizeRedemptionGatewayAdapter.sol
- integrations-v3/contracts/helpers/securitize/SecuritizeLiquidator.sol
- integrations-v3/contracts/helpers/securitize/SecuritizeRedeemer.sol
- integrations-v3/contracts/helpers/securitize/SecuritizeRedemptionGateway.sol
- integrations-v3/contracts/helpers/securitize/SecuritizeRedemptionPhantomToken.sol
- periphery-v3/contracts/DefaultKYCUnderlying.sol

- periphery-v3/contracts/SecuritizeDegenNFT.sol
- periphery-v3/contracts/SecuritizeKYCFactory.sol
- periphery-v3/contracts/SecuritizeWallet.sol

2.3. Threat Model

The assessment presumes actions of an intruder who might have capabilities of any role (an external user, token owner, token service owner, a contract). The centralization risks have not been considered upon the request of the Customer.

The main possible threat actors are:

- User,
- Protocol owner,
- Liquidity Token owner/contract.

2.4. Weakness Scoring

An expert evaluation scores the findings in this report, an impact of each vulnerability is calculated based on its ease of exploitation (based on the industry practice and our experience) and severity (for the considered threats).

2.5. Disclaimer

Due to the intrinsic nature of the software and vulnerabilities and the changing threat landscape, it cannot be generally guaranteed that a certain security property of a program holds.

Therefore, this report is provided “as is” and is not a guarantee that the analyzed system does not contain any other security weaknesses or vulnerabilities. Furthermore, this report is not an endorsement of the Customer’s project, nor is it an investment advice.

That being said, Decurity exercises best effort to perform their contractual obligations and follow the industry methodologies to discover as many weaknesses as possible and maximize the audit coverage using the limited resources.

3. Summary

As a result of this work, we have discovered a single high security issue.

The other suggestions included fixing the low-risk issue.

The team has given the feedback for the suggested changes and explanation for the underlying code.

3.1. Suggestions

The table below contains the discovered issues, their risk level, and their status as of May 8, 2026.

Table. Discovered weaknesses

Issue	Contract	Risk Level	Status
Balance position valuation diverge for unclaimed redeemers	integrations-v3/contracts/helpers/securitize/SecuritizeLiquidator.sol	**High**	Fixed
Missing redemptionGateway validation in liquidatePendingRedemption	integrations-v3/contracts/helpers/securitize/SecuritizeLiquidator.sol	**Info**	Fixed

4. General Recommendations

This section contains general recommendations on how to improve overall security level.

The Findings section contains technical recommendations for each discovered issue.

4.1. Security Process Improvement

The following is a brief long-term action plan to mitigate further weaknesses and bring the product security to a higher level:

- Keep the whitepaper and documentation updated to make it consistent with the implementation and the intended use cases of the system,
- Perform regular audits for all the new contracts and updates,
- Ensure the secure off-chain storage and processing of the credentials (e.g. the privileged private keys),
- Launch a public bug bounty campaign for the contracts.

5. Findings

5.1. Balance position valuation diverge for unclaimed redeemers

Risk Level: High

Status: Fixed in the commit.

Contracts:

- integrations-v3/contracts/helpers/securitize/SecuritizeLiquidator.sol

Description:

The liquidator and the phantom token use different valuation formulas for pending redeemers. This creates an exploitable gap whenever NAV drops between Securitize processing the redemption and the liquidation.

Phantom token (used by `creditFacade` for collateral and health checks):

```
// SecuritizeRedeemer.getRedemptionAmount()
uint256 maxRedemptionAmount = pendingDsTokenAmount *
stableCoinDecimalsMultiplier * startingNavRate
    / (dsTokenDecimalsMultiplier * dsTokenDecimalsMultiplier);
uint256 currentRedemptionAmount = getCurrentRedemptionValue();
uint256 redemptionAmount =
    currentRedemptionAmount > maxRedemptionAmount ? maxRedemptionAmount :
currentRedemptionAmount;
return currentBalance > redemptionAmount ? currentBalance :
redemptionAmount;
```

`max(currentBalance, min(starting * pending, current * pending))`

Liquidator (used to derive `underlyingAmount` payment):

```
// SecuritizeLiquidator._calcCollateralAndLiquidityValues
collateralValue +=
SecuritizeRedeemer(redeemers[i]).getCurrentRedemptionValue();
```

`current * pending` (no balance floor, no min cap)

These valuations diverge when `currentBalance > current × pending`. This can happen when Securitize settles the redeemer at `settlementNAV ≥ currentNAV` and NAV drops before liquidation. It can also happen when anyone, including the borrower, transfers stablecoins directly to the redeemer.

Setup a CreditAccount with two redeemers:

R1: settled (or self-funded) such that `balanceOf(R1) > pending₁ × currentNAV`. `getRedemptionAmount` falls into the `max(balance, ...)` branch. The phantom-side view reports the high balance, while the liquidator-side view reports the low `current × pending`.

R2: an unsettled large chunk (`balance(R2) = 0, pending₂ ≫ 0`).

Position is HF<1 (NAV drift, accrued interest, etc.).

This setup creates a divergence between two views of R1 used by different code paths.

Path 1: `liquidatePendingRedemption` reverts on `minRemainingFunds`.

`SecuritizeLiquidator` computes the underlying amount the liquidator is forced to pay using the low liquidator-side view, then transfers exactly that amount and approves it to the credit manager:

```
// SecuritizeLiquidator.sol::liquidatePendingRedemption
(uint256 collateralValue, uint256 liquidityAmount) =
_calcCollateralAndLiquidityValues(
    creditAccount, creditManager, underlying, redemptionGateway, redeemers,
liquidationDiscount
);
 underlyingAmount = collateralValue * liquidationDiscount /
PERCENTAGE_FACTOR; // ← LOW view × discount
 ...
 IERC20(underlying).safeTransferFrom(msg.sender, address(this),
underlyingAmount);
IERC20(underlying).forceApprove(creditManager, underlyingAmount);
 isTransferAllowed = true;
ICreditFacadeV3(creditFacade).liquidateCreditAccount(creditAccount,
creditAccount, calls, lossPolicyData);
```

Inside the call, CreditManager evaluates `minRemainingFunds` from `cdd.totalValue`, which is the high view. The phantom balance reads through `getRedemptionAmount()` and falls into the `max(balance, ...)` branch for R1:

```
// CreditManagerV3.sol:345
if (remainingFunds < minRemainingFunds) {
    revert InsufficientRemainingFundsException();
}
```

Because `totalValue` is built against the inflated phantom, `minRemainingFunds` is sized for the high view and the call reverts with `InsufficientRemainingFundsException`. The liquidator has no way to increase `underlyingAmount`. It is hardcoded inside the function and derived from the same low-side `collateralValue` that creates the shortfall.

```
// CreditLogic.sol:80-91
uint256 totalFunds = totalValue * liquidationDiscount / PERCENTAGE_FACTOR;
amountToPool += totalValue * feeLiquidation / PERCENTAGE_FACTOR;
...
if (totalFunds > amountToPoolWithFee) {
    remainingFunds = totalFunds - amountToPoolWithFee; // minRemainingFunds
} else /* bad-debt — minRemainingFunds = 0 */
```

Path 2: standard `CreditFacadeV3.liquidateCreditAccount` — R2 makes it uneconomic.

If R1 were the only redeemer, the liquidator could skip `SecuritizeLiquidator` entirely and call `CreditFacadeV3.liquidateCreditAccount` directly with a multicall containing `claim(R1)`, transferring R1's stables onto the CA as underlying. That alone would give the CA enough underlying to satisfy `amountToPool`, and the liquidator would walk away with `amountToLiquidator` as a normal liquidation premium.

R2 breaks this workaround.

`cdd.totalValue` is computed pre-multicall and includes R2's phantom value. `minRemainingFunds` is sized against the full `totalValue`, including R2.

To pass `actualRemaining ≥ minRemainingFunds`, the liquidator must leave on the CA underlying worth approximately `totalValue × discount`. This means the liquidator must fund R2's portion of the debt out of pocket by covering an extra ~$\text{pending}_2 \times \text{current} \times \text{discount}$ of underlying through `addCollateral`. However, `transferRedeemer` is gated to `SecuritizeLiquidator`, so the standard path cannot reassign R2 to the liquidator.

So the standard path forces the liquidator to either:

- pay full underlying for R2's pending value and never recover it
- skip claim(R2) and still fail minRemainingFunds, since totalValue includes R2

In every variant, the liquidator is net-negative on R2's value.

As a result, the CA can sit at `HF < minHF` indefinitely. Each time R1 (or any other redeemer) settles via Securitize, the borrower can spawn a fresh unsettled redeemer (up to the 10-cap) and re-fund any redeemer with stables to keep both paths blocked.

Additional impact: under-payment when CA has an underlying buffer

Another impact occurs when the CA holds additional underlying on top of the redeemer claims.

In this case, `minRemainingFunds` passes and `liquidatePendingRedemption` runs to completion. The liquidator pays:

```
underlyingAmount = collateralValue * liquidationDiscount /
PERCENTAGE_FACTOR;
```

where `collateralValue` is summed from `getCurrentRedemptionValue()` (low view). The fair amount, computed from `getRedemptionAmount()` (high view, what the borrower's collateral is actually worth), would be larger by `divergence × liquidationDiscount`.

Setup: $100k DS position split into R1 (10k DS, settled at NAV 1.05 → balance $10 500) and R2 (90k DS, unsettled, balance 0). Borrower keeps 36k underlying as a buffer on the CA. Phantom quota rate is raised to 50%/yr. After ~6 months of pure quota-interest accrual, the position drifts to HF<1 while staying non-bad-debt.

```
debt = 86 000 // principal borrowed
totalValue = 132 000 // 96k phantom + 36k underlying buffer on CA
totalDebt = 122 695 // debt + accruedInterest + accruedFees, ~6 months at
50%/yr quota rate
totalFunds = 132 000 × 0.96 = 126 720 // totalValue × liquidationDiscount
amountToPool = 122 695 + 132 000 × 0.015 = 124 675 // totalDebt + totalValue
× feeLiquidation
minRemainingFunds = 126 720 − 124 675 = 2 045 // totalFunds − amountToPool
 collateralValue = 95 000 // Σ getCurrentRedemptionValue per redeemer = 0.95
× 100k pending
underlyingAmount = 91 200 // collateralValue × discount, what SL pulls from
the liquidator
fairUnderlyingAmount = 92 160 // phantom 96k × discount, what would be paid
against the high view
 extraction = 92 160 − 91 200 = 960 // (phantom − collateralValue) ×
discount
```

The borrower's recoverable residual on the CA after `closeCreditAccount` is 960 underlying smaller than under fair pricing.

Remediation:

Consider using similar position valuation math for phantom token and liquidator.

5.2. Missing redemptionGateway validation in liquidatePendingRedemption

Risk Level: Info

Status: Fixed in the commit.

Contracts:

- integrations-v3/contracts/helpers/securitize/SecuritizeLiquidator.sol

Description:

`liquidatePendingRedemption` accepts `redemptionGateway` as a user-supplied parameter and validates it only via a self-attested check:

```
if (ISecuritizeRedemptionGateway(redemptionGateway).transferMaster() !=
address(this)) {
    revert NotValidGatewayException();
}
```

An attacker can deploy a `MaliciousGateway` whose `transferMaster()` returns `address(SecuritizeLiquidator)` and supply it.

The check passes, and the rest of the gateway interface (`getUnclaimedRedeemers`, `dsToken`, `stableCoinToken`) becomes attacker-controlled.

Fortunately, no exploitable impact is reachable from this in practice.

Remediation:

Consider adding an upfront check that the supplied gateway has an adapter registered in the credit manager:

```
address creditManager = ICreditAccountV3(creditAccount).creditManager();
if (ICreditManagerV3(creditManager).contractToAdapter(redemptionGateway) ==
address(0)) {
    revert NotValidGatewayException();
}
```

6. Appendix

6.1. About us

The Decurity team consists of experienced hackers who have been doing application security assessments and penetration testing for over a decade.

During the recent years, we've gained expertise in the blockchain field and have conducted numerous audits for both centralized and decentralized projects: exchanges, protocols, and blockchain nodes.

Our efforts have helped to protect hundreds of millions of dollars and make web3 a safer place.